Clase 14.

Chavez Aquiagual Rodrigo

Sistemas Distribuidos

Profesor: Ukranio Coronilla

Contreras Grupo 7CM4

Ejercicio 1.

¿Qué cree que sucedería con los tiempos de respuesta promedio del servidor si en

la línea:

server.setExecutor(Executors.newFixedThreadPool(8));

se cambia el ocho por un uno?

Yo esperaría que al cambiar el número de hilos del pool de 8 a 1, el servidor pasará de poder atender hasta 8 peticiones simultáneamente a atenderlas de una en una, por lo que si llegan varias solicitudes al mismo tiempo, se encolarán y se procesarán secuencialmente, aumentando el tiempo de respuesta de las que quedan en espera.

Ejecución:

Para el servidor con un pool de 1 hilo:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ time curl -X POST -d
 "2,3,4" http://localhost:8082/task
El resultado de la multiplicación es 24

real    0m5.108s
user    0m0.004s
sys     0m0.011s
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$
```

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ time curl -X POST -d
 "2,3,4" http://localhost:8082/task
El resultado de la multiplicación es 24

real    0m8.666s
user    0m0.007s
sys     0m0.009s
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$
```

Aqui la primera petición ocupa el primer hilo y empieza a realizarse, pero la segunda petición se queda encolada hasta que la primer petición termine, es por eso que al final se acumulan los tiempos de las dos peticiones.

Ejecución:

Para el servidor con un pool de 8 hilos:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ time curl -X POST -d
 "2,3,4" http://localhost:8080/task
El resultado de la multiplicación es 24

real    0m5.020s
user    0m0.004s
sys     0m0.011s
```

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ time curl -X POST -d
 "2,3,4" http://localhost:8080/task
El resultado de la multiplicación es 24

real    0m5.018s
user    0m0.006s
sys     0m0.010s
```

Lo que está sucediendo es que aquí ambas peticiones se están procesando en paralelo porque hay hilos disponibles, por lo que no se acumulan los tiempos de espera, cada uno se hace simultaneamente.

Ejercicio 2.

Código:

```
// Imprimir información del cuerpo
System.out.println("Recibida solicitud POST en /task. Bytes: " + requestBytes.length);
System.out.println("Contenido: " + new String(requestBytes));
```

Ejecución:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ curl --data "1756677,78788" localhost:8080
/task
El resultado de la multiplicación es 138405067476
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ curl --data "1756677,787088" localhost:808
0/task
El resultado de la multiplicación es 1382659386576
```

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ java WebServer 8080
Servidor escuchando en el puerto 8080
Recibida solicitud POST en /task. Bytes: 13
Contenido: 1756677,78788
Recibida solicitud POST en /task. Bytes: 14
Contenido: 1756677,787088
```

Ejercicio 3.

Código:

```
System.out.println("Headers recibidos en /task: " + headers.size());
for (String key : headers.keySet()) {
    System.out.println(key + ": " + headers.get(key));
}
```

Ejecución:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ java WebServer 8080
Servidor escuchando en el puerto 8080
Headers recibidos en /task: 5
Accept: [*/*]
Host: [localhost:8080]
User-agent: [curl/8.5.0]
Content-type: [application/x-www-form-urlencoded]
Content-length: [13]
Recibida solicitud POST en /task. Bytes: 13
Contenido: 1756677,78788
```

Ejercicio 4.

Código:

```java
    long finishTime = System.nanoTime();

    if (isDebugMode) {
        long elapsed = finishTime - startTime;
        long seconds = elapsed / 1_000_000_000;
        long millis = (elapsed % 1_000_000_000) / 1_000_000;
        String debugMessage = String.format("La operacion tomo %d nanosegundos = %d segundos con %d milisegundos.",
                elapsed, seconds, millis);
        // También se imprime en consola para facilitar la depuración
        System.out.println(debugMessage);
        exchange.getResponseHeaders().put("X-Debug-Info", Arrays.asList(debugMessage));
    }

    sendResponse(responseBytes, exchange);
}
```

Ejecución:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ curl -v --header "X-Debug:true" --da
ta "50,100" localhost:8080/task
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> POST /task HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.5.0
> Accept: */*
> X-Debug:true
> Content-Length: 6
> Content-Type: application/x-www-form-urlencoded
>
< HTTP/1.1 200 OK
< Date: Fri, 06 Mar 2026 00:09:46 GMT
< X-debug-info: La operacion tomo 6860496 nanosegundos = 0 segundos con 6 milisegundos.
< Content-length: 43
<
El resultado de la multiplicación es 5000
* Connection #0 to host localhost left intact
```

Ejercicio 5.

Código agregado:

```java
    private void handleSearchTokenRequest(HttpExchange exchange) throws IOException {
        // Solo aceptamos POST
        if (!exchange.getRequestMethod().equalsIgnoreCase("post")) {
            exchange.close();
            return;
        }

        Headers headers = exchange.getRequestHeaders();

        // Verificar si modo debug está activado
        boolean isDebugMode = false;
        if (headers.containsKey("X-Debug") && headers.get("X-Debug").get(0).equalsIgnoreCase("true")) {
            isDebugMode = true;
        }

        long startTime = System.nanoTime();

        // Leer cuerpo de la petición
        byte[] requestBytes = exchange.getRequestBody().readAllBytes();
        String body = new String(requestBytes).trim();

        // Parsear: se espera "n,subcadena" (ej. "34567890,SOL")
        String[] parts = body.split(",");
        if (parts.length != 2) {
            String error = "Formato inválido. Use: <número>,<tres letras>\n";
            sendResponse(error.getBytes(), exchange);
            return;
        }

        int numTokens;
        try {
            numTokens = Integer.parseInt(parts[0].trim());
        } catch (NumberFormatException e) {
            String error = "El primer parámetro debe ser un número entero.\n";
```

```java
            sendResponse(error.getBytes(), exchange);
            return;
        }

        String target = parts[1].trim();
        if (target.length() != 3) {
            String error = "La subcadena debe tener exactamente 3 letras.\n";
            sendResponse(error.getBytes(), exchange);
            return;
        }

        // Generar la cadenota
        char[] cadenota = new char[numTokens * 4]; // cada token: 3 letras + espacio
        Random rand = new Random();

        for (int i = 0; i < numTokens; i++) {
            int base = i * 4;
            // Letras mayúsculas aleatorias (A-Z)
            cadenota[base] = (char) (rand.nextInt(26) + 'A');
            cadenota[base + 1] = (char) (rand.nextInt(26) + 'A');
            cadenota[base + 2] = (char) (rand.nextInt(26) + 'A');
            cadenota[base + 3] = ' '; // espacio separador
        }

        // Buscar la subcadena objetivo
        int count = 0;
        char c0 = target.charAt(0);
        char c1 = target.charAt(1);
        char c2 = target.charAt(2);

        for (int i = 0; i < cadenota.length - 2; i++) {
            if (cadenota[i] == c0 && cadenota[i + 1] == c1 && cadenota[i + 2] == c2) {
                count++;
            }
```

```java
        // Preparar respuesta
        String responseMessage = "Total de apariciones: " + count + "\n";
        byte[] responseBytes = responseMessage.getBytes();

        long finishTime = System.nanoTime();

        // Si modo debug, agregar header con tiempo
        if (isDebugMode) {
            long elapsed = finishTime - startTime;
            long seconds = elapsed / 1_000_000_000;
            long millis = (elapsed % 1_000_000_000) / 1_000_000;
            String debugMessage = String.format("La operacion tomo %d nanosegundos = %d segundos con %d milisegundos.",
                    elapsed, seconds, millis);
            exchange.getResponseHeaders().put("X-Debug-Info", Arrays.asList(debugMessage));
        }

        sendResponse(responseBytes, exchange);
    }
```

Ejecución:

```
rodri@rodri-VivoBook-ASUS-Laptop-X505ZA-X505ZA:~/Downloads/Server$ time curl -v --header 'X-Debug:true'
 --data '34567890,SOL' localhost:8080/searchtoken
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> POST /searchtoken HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.5.0
> Accept: */*
> X-Debug:true
> Content-Length: 12
> Content-Type: application/x-www-form-urlencoded
>
< HTTP/1.1 200 OK
< Date: Fri, 06 Mar 2026 00:23:26 GMT
< X-debug-info: La operacion tomo 2086570456 nanosegundos = 2 segundos con 86 milisegundos.
< Content-length: 27
<
Total de apariciones: 1982
```